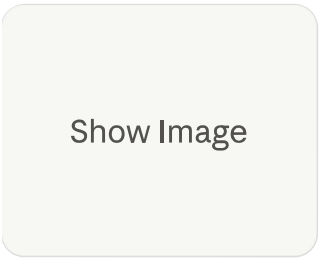
Hands-on Tutorials

Which Local Coding Model Is Worth Running on Your Own GPU?

I tested 17 local models on one 16 GB card with 164 coding problems. One 20B model stood out, and the reasons are more interesting than the score.



Quality versus speed for local models on one 16 GB GPU. Each point is the largest context window that still runs at a usable speed. Image created by author.

Every week a new model arrives with a leaderboard screenshot. The numbers were measured in a data center, on a GPU I do not own, at full precision. My question is simpler: which model should I actually run on my own machine, with a context window I can use, at a speed I can work with?

To answer it, I ran 17 local models through the same 164 coding problems, at context windows from 50k to 180k tokens, and I measured three things on every run: how often the code passed the tests, how fast the model decoded, and how much GPU memory the window cost. In this blog I will walk you through how to set that test up, what the 164 problems actually are, and what the results say for practical use. I will also explain why GPT-OSS-20B came out on top, and where the minimum speed for practical coding really sits.

This is a practical guide. The goal is a method you can rerun every time a new model lands, and a clear answer for the models that matter on a desk.

If you found this article helpful, you are welcome to **follow me** because I write more about data science and local AI! Try the hands-on examples in this blog. This will help you to learn quicker, understand better, and remember longer. Grab a coffee and have fun!

Disclosure: I have no affiliation with any of the model vendors mentioned in this blog.

. . .

Why a model card is not a benchmark of your machine

A model card describes a model running on a data center GPU at full precision. Running it locally changes three things at once.

First, **quantization**. To make a model fit, you store its weights with fewer bits. That saves memory but may cost accuracy. Second, **offloading**. When the weights do not fit in GPU memory, some layers move to system RAM, which is far slower to read from. Third, **the KV cache**. The memory that holds the conversation grows with every token of context, so the context window printed on the box and the context window you can use are two different numbers.

Let me elaborate on why these three matter together. Each one shows up in a different metric: accuracy for quantization, tokens per second for offloading, and GPU memory for context. A single leaderboard number hides all of that. *This is exactly why "should I download this model?" cannot be answered from a model card.*

In the next section we look at the test itself, because the test decides what you can conclude.

...

Step 1: The 164 HumanEval problems

Every score in this blog comes from HumanEval, a benchmark of 164 hand-written Python programming problems released by OpenAI in 2021. Each problem gives the model a function signature and a docstring, and asks it to write the body. Here is a simplified example of the format:

```python
def has_close_elements(numbers: list[float], threshold: float) -> bool:
    """Check if in given list of numbers, are any two numbers closer to each
    other than given threshold.
    >>> has_close_elements([1.0, 2.0, 3.0], 0.5)
    False
    >>> has_close_elements([1.0, 2.8, 3.0, 4.0, 5.0, 2.0], 0.3)
    True
    """
```

The model's job is to write the code that goes under the docstring. The problems cover string handling, list and number puzzles, simple algorithms, and some basic math. Each one comes with a hidden test function. The test calls the generated function on several inputs and checks the outputs, so there is no partial credit: the code either passes every check or it fails.

Let me elaborate on what this means for the numbers. The 164 problems are short. The prompts average about 180 tokens, and the answers are usually a few lines. That makes HumanEval a good test of whether a model can write correct small functions, and a poor test of whether it can work inside a large project. It is also old enough that many models have seen it during training. *This is exactly why I read the scores as a ranking of small coding skill, not as a verdict on a model's usefulness in general.*

Two facts about the benchmark matter for everything below. First, a pass rate on 164 problems has a real margin of error: a 90% score carries a 95% confidence interval about 9 points wide. Second, the test runs the generated code, so a wrong answer that crashes counts the same as one that returns a wrong value. The scoring script runs each answer in a temporary directory and checks the result.

In the next section we set up the benchmark that runs these problems through a local model.

...

Step 2: Will it fit on your card?

Before downloading anything, estimate the memory a model needs. Weights take `parameters × bits per weight / 8` bytes. The KV cache takes `context × bytes per token`. Add about a gigabyte for buffers and you have a rough budget.

The "bits per weight" in a quantization name is not the number in the name. A file called Q4_K_M averages about 4.8 bits per weight, because some tensors are kept at higher precision. Here is a small estimator:

```python
BITS = {"Q3_K_M": 3.9, "Q4_K_M": 4.8, "Q5_K_M": 5.7, "Q6_K": 6.6, "Q8_0": 8.5}

def estimate_vram(params_b, bits, context, kv_bytes_per_token, overhead_gb=1.0):
    weights_gb = params_b * bits / 8
    kv_gb = context * kv_bytes_per_token / 1e9
    return weights_gb, kv_gb, weights_gb + kv_gb + overhead_gb

# A 27B model with a 32k context window and 54,000 bytes of KV cache per token.
# Replace the last number with the value measured for your model in Step 4.
for name, bits in BITS.items():
    w, kv, total = estimate_vram(27, bits, 32_000, 54_000)
    print(f"{name:7s} weights {w:5.1f} GB | KV {kv:4.1f} GB | total {total:5.1f} GB
```

```
Q3_K_M  weights  13.2 GB | KV  1.7 GB | total  15.9 GB
Q4_K_M  weights  16.2 GB | KV  1.7 GB | total  18.9 GB
Q5_K_M  weights  19.2 GB | KV  1.7 GB | total  22.0 GB
Q6_K    weights  22.3 GB | KV  1.7 GB | total  25.0 GB
Q8_0    weights  28.7 GB | KV  1.7 GB | total  31.4 GB
```

My card has about 16 GB, and `nvidia-smi` never showed more than 15.9 GB in use during any run. So a 27B model at 4-bit does not fit, and even 3-bit is borderline. The logs agree: the highest reading in all of my runs came from a 27B model at 3-bit. *Note that the `nvidia-smi` readings are in MiB, which I quote as GB for readability, so treat them as approximate.*

Memory is only half of the estimate. Decoding speed is limited by how fast the hardware can read the weights for each token:

python

```python
def decode_ceiling(active_params_b, bits, bandwidth_gb_s):
    """Upper bound on tokens/s if memory bandwidth is the only limit."""
    return bandwidth_gb_s / (active_params_b * bits / 8)

# Illustrative bandwidth of 400 GB/s. Replace it with your card's number.
print(decode_ceiling(27, 4.8, 400))   # dense 27B:                      24.7 tok/s
print(decode_ceiling(3, 4.8, 400))    # mixture-of-experts, 3B active: 222.2 tok/s
```

This is the trick behind many fast local models. A mixture-of-experts model such as a "20B-A3B" stores 20 or 30 billion parameters but reads only about 3 billion for each token. A dense model reads all of its weights for every token, so it is slow on any card.

Before you measure anything, you can already predict that a dense model that does not fit will be several times slower than a sparse model that does.

In the next section we turn the estimate into a measured benchmark.

...

Step 3: Build the benchmark

The benchmark is one Python script that talks to LM Studio. For every model and every context length, it does four things:

1. Unload everything, then load the model through LM Studio's REST API with the requested context length.

2. Stream an answer for each of the 164 HumanEval problems, timing the first token and the decoding speed.
3. Run the generated code against the official tests in a temporary directory.
4. Append one row to a CSV file and flush it to disk immediately.

Install the dependencies and start the LM Studio server:

```bash
pip install openai pandas requests datasets matplotlib scipy
```

The model load is a single request, and the context length is the variable we sweep:

```python
payload = {
    "model": model,
    "context_length": context,
    "flash_attention": True,
    "offload_kv_cache_to_gpu": True,
    "echo_load_config": True,   # LM Studio returns the settings it applied
}
r = requests.post(f"{base_url}/api/v1/models/load", json=payload, timeout=900)
```

The `echo_load_config` flag matters more than it looks. LM Studio may not give you exactly what you asked for, so the script stores the returned configuration in a diagnostics file next to the results.

My first version had a bug that inflated every score, and it is worth knowing about because it is easy to repeat. HumanEval's `test` field only *defines* a function called `check`. It never calls it. My script ran the generated code together with that definition, so any answer that did not crash counted as a pass. The fix is one line:

```python
def build_program(code, row):
    prompt, entry = row["prompt"], row["entry_point"]
    # ... stitch the prompt (imports, helpers) and the model's code together ...
    return body + "\n\n" + row["test"] + f"\n\ncheck({entry})\n"   # the missing ca
```

This is exactly why you should sanity check a benchmark with a deliberately wrong solution before you trust a single number.

A real run looks like this:

```bash
python llm_coding_benchmark.py \
    --models "openai/gpt-oss-20b" \
    --contexts 50000 80000 100000 120000 131072 \
    --max-tokens 8192
```

Local runs get interrupted. Laptops sleep, servers crash, and sometimes you press Ctrl-C because the model is too slow. Every finished problem is already in the CSV, so running the same command again resumes where it stopped, and finished configurations do not reload the model:

```
Already complete (164/164) -> skipping, model will not be loaded.
Resuming: 87/164 already done.
```

Failed requests are retried on the next run, and a torn last row after a hard kill is repaired automatically. All results below come from runs with short prompts (about 150 to 270 tokens) inside a large, mostly empty window, which isolates what the window itself costs in speed and memory. *In the next section we look at the results, starting with the models that are worth running at all.*

...

Step 4: Which models deserve a place on your desk?

Here are all 17 models that finished at least one run. Speed is the decoding speed at the smallest window, and the largest window is the one before the speed collapses, if it does.

Model	Pass rate (95% CI)	Speed (smallest window)	Speed (largest window)	Truncated failures	GPU peak
GPT-OSS-20B	90.2% (84.7 to 93.9)	262.3 tok/s	255.6 tok/s (131k)	0 of 16	14.9 GB
ornith-1.0-9B	89.6% (84.0 to 93.4)	62.7 tok/s	62.9 tok/s (180k)	5 of 17	12.9 GB
Qwen3-Coder-30B-A3B	87.8% (81.9 to 92.0)	141.6 tok/s	141.5 tok/s (120k)	1 of 20	15.7 GB
Gemma-4-12B-coder	87.8% (81.9 to 92.0)	44.5 tok/s	44.7 tok/s (131k)	0 of 20	13.7 GB
Qwen3.6-35B-A3B	86.6% (80.5 to 91.0)	37.1 tok/s	37.0 tok/s (180k)	21 of 22	14.2 GB
Qwen2.5-Coder-3B	83.5% (77.1 to 88.4)	156.5 tok/s	156.5 tok/s (32k)	0 of 27	3.7 GB
LFM2-24B-A2B	82.9% (76.4 to 87.9)	135.7 tok/s	137.0 tok/s (128k)	0 of 28	14.8 GB
qwythos-9B	82.9% (76.4 to 87.9)	100.7 tok/s	99.4 tok/s (180k)	5 of 28	15.8 GB
claude3.5-4B	77.4% (70.5 to 83.2)	131.1 tok/s	135.8 tok/s (80k)	0 of 37	15.8 GB
Codestral-22B	76.2% (69.2 to 82.1)	1.2 tok/s	1.2 tok/s (32k)	0 of 39	15.7 GB
GLM-4.7-flash	67.1% (59.6 to 73.8)	68.0 tok/s	67.8 tok/s (100k)	25 of 54	15.6 GB
Gemma-4-26B-A4B	54.9% (47.2 to 62.3)	51.8 tok/s	51.5 tok/s (180k)	74 of 74	15.8 GB
GLM-4.7-flash-REAP	1.8% (0.6 to 5.2)	57.1 tok/s	57.1 tok/s (50k)	169 of 176	15.2 GB

Pass rates are from full runs of 164 problems. The GPU memory is the highest reading across all windows.

Read the table in two passes. In the first pass, drop everything below 80% pass rate. That leaves eight models, and their scores are not identical. Where two models have similar scores, a 95% interval about 9 points wide cannot tell them apart. Speed then decides, and the picture gets much clearer.

Now the second pass, and the question this blog is about: which of the eight can you run at a usable speed with a context window you actually need? These are the candidates:

Model	Pass rate (95% CI)	Decode speed	Largest window at speed	Speed there	Speed cliff	GPU memory (peak)
GPT-OSS-20B	90.2% (84.7 to 93.9)	262.3 tok/s	131k	255.6 tok/s	none up to 131k	14.9 GB
ornith-1.0-9B	89.6% (84.0 to 93.4)	62.7 tok/s	180k	62.9 tok/s	none up to 180k	12.9 GB
Qwen3-Coder-30B-A3B	87.8% (81.9 to 92.0)	141.6 tok/s	120k	141.5 tok/s	yes, at 150k	15.7 GB
Gemma-4-12B-coder	87.8% (81.9 to 92.0)	44.5 tok/s	131k	44.7 tok/s	none up to 131k	13.7 GB
Qwen3.6-35B-A3B	86.6% (80.5 to 91.0)	37.1 tok/s	180k	37.0 tok/s	none up to 180k	14.2 GB
Qwen2.5-Coder-3B	83.5% (77.1 to 88.4)	156.5 tok/s	32k	156.5 tok/s	none up to 32k	3.7 GB
LFM2-24B-A2B	82.9% (76.4 to 87.9)	135.7 tok/s	128k	137.0 tok/s	none up to 128k	14.8 GB
qwythos-9B	82.9% (76.4 to 87.9)	100.7 tok/s	180k	99.4 tok/s	none up to 180k	15.8 GB

Speed cliff: the first window where decoding fell below half of its small-window speed (or below 10 tok/s). The largest window is the last one before that cliff.

Three things stand out.

First, the top two are not close. GPT-OSS-20B is 0.6 points ahead of ornith-1.0-9B on score, which is far inside the margin of error, but it decodes at 255 tok/s against 63 tok/s, about four times as fast. The speed gap is real, and the score gap is noise.

Second, speed and window size do not travel together. Qwen3-Coder-30B-A3B is the second-fastest model in this group at 141 tok/s, but it falls off a cliff at 150k, and its memory is at the card limit. ornith and Qwen3.6 run at 180k without any cliff, but they are much slower.

Third, the table has one clear loser. The two GLM variants and the Gemma 26B model score too low to be useful at any speed. The REAP variant of GLM-4.7-flash is the worst result in the whole study: 169 of its 176 failures were answers cut off by the token budget, and it passed only 3 of 179 runs. That is not a capability measurement, but on a desk it is no tool at all.

A model is a candidate only if it is accurate and fast at the context you need. Score alone is not enough, and speed alone is not enough.

In the next section we look at the model that comes out on top.

. . .

Step 5: Why GPT-OSS-20B wins

GPT-OSS-20B combines three things that rarely appear together in a local model: a high score, the fastest decoding in the study, and a context window it can use without a speed cliff. Here is its full result at each window:

Context window	Pass rate	Decode speed	Time to first token	GPU memory
50k	90.2%	262.3 tok/s	2.95 s	13.0 GB
80k	91.5%	241.6 tok/s	2.82 s	13.6 GB
100k	90.2%	258.0 tok/s	2.96 s	14.1 GB
120k	90.2%	254.7 tok/s	2.97 s	14.6 GB
131k	90.2%	255.6 tok/s	2.97 s	14.9 GB

It passed 148 to 150 of the 164 problems at every window, which puts it at the top of the score table. It decodes at 242 to 262 tok/s at every window, which is about ten times what a dense 27B model would manage on this card by the bandwidth estimate in Step 2. It never truncated an

answer, so every one of its 14 to 16 failures was a real wrong answer. And its memory grows slowly with the window: 13.0 GB at 50k, 14.9 GB at 131k, still with room to spare.

Let me elaborate on why it is efficient, because efficiency is what matters on a desk. GPT-OSS-20B is a mixture-of-experts model, so each token reads only a small part of its weights. Its attention keeps the KV cache small enough that the 131k window costs about 2 GB more than the 50k window. A dense model of similar size would have to read everything for every token, and it would run out of memory long before 131k. *This is exactly why the model card number "20B" tells you little. What matters is how many parameters are read per token and how much memory each token of context costs.*

Here is what that means in practice. The 131k window is the longest one I tested with GPT-OSS-20B, and the test used only short prompts. Real use, with a whole file or a long conversation in the window, will fill more of that memory. The speed holds at 131k with an empty window, but I did not measure a full window, so treat the 131k result as a ceiling for the memory cost and not a guarantee for the speed when the window is full. The probe mode in the benchmark is designed for exactly that test.

For the same reason, I recommend it as the default local coding model on a card like mine. It gives the best combination of accuracy and speed, and it leaves memory for the context you actually use.

. . .

Step 6: How slow is too slow?

Codestral-22B scored 76% but ran at 1.2 tok/s. At that speed, one 250-token answer takes about three and a half minutes, and the 164 problems take nearly nine and a half hours. That is the moment to ask a simple question: what is the minimum decoding speed that still makes sense for coding?

The answer depends on how you use the model. Speed decides how long you wait for each answer, and accuracy decides how many of those answers you can use. Combine them, and you get the number that matters: solutions per hour, which is the time per correct answer inverted. Here is the math for a 250-token answer, using a 90% pass rate and the time to first token from GPT-OSS:

```
python
```

```python
def solutions_per_hour(decode_tps, pass_rate, ttft=2.9, answer_tokens=250, test_tim
    seconds_per_answer = ttft + answer_tokens / decode_tps + test_time
    seconds_per_correct = seconds_per_answer / pass_rate
    return 3600 / seconds_per_correct

for tps in (5, 10, 20, 40, 100, 250):
    print(f"{tps:>4} tok/s -> {solutions_per_hour(tps, 0.90):5.0f} solutions per ho
```

```
   5 tok/s ->    61 solutions per hour
  10 tok/s ->   116 solutions per hour
  20 tok/s ->   209 solutions per hour
  40 tok/s ->   350 solutions per hour
 100 tok/s ->   589 solutions per hour
 250 tok/s ->   810 solutions per hour
```

Decode speed	One 250-token answer	Per correct answer (90% pass)	Solutions per hour	Verdict
5 tok/s	50.0 s	58.9 s	61	too slow: a coffee per function
10 tok/s	25.0 s	31.1 s	116	floor for batch work
20 tok/s	12.5 s	17.2 s	209	comfortable for interactive use
40 tok/s	6.2 s	10.3 s	350	fast
100 tok/s	2.5 s	6.1 s	589	very fast
250 tok/s	1.0 s	4.4 s	810	GPT-OSS-20B on this card

The answer I use is this. At 10 tok/s, a single 250-token answer takes 25 seconds. That is the floor: below it, you spend more time waiting than thinking, and I would only run such a model for overnight batch jobs. At 20 tok/s a function appears in about 12 seconds, which is comfortable for interactive coding, because you can read along while it writes. Anything above 40 tok/s feels instant, and past 100 tok/s the bottleneck moves from the model to you.

Two caveats matter here. The 250-token answer is an assumption, and thinking models write more before the code, so their real answer time is longer. The 10 tok/s floor is for one request at a time, and an agent that makes many calls in a row needs more. For those cases, I would plan for 20 tok/s or more.

Speed is a filter, not just a metric. Below about 10 tok/s, a model is a batch tool; above 20 tok/s, it can be a coding partner.

...

Rules of thumb for local models

After running this on many releases, these are the rules I follow:

1. Do the memory arithmetic before the download. If the weights and the KV cache exceed your VRAM, expect the speed to collapse rather than degrade gently.
2. Prefer sparse models on a small card. GPT-OSS-20B and the Qwen3-Coder 30B-A3B are fast because they read only a few billion parameters per token.
3. Test the window you need, not the window on the box. Speed and memory are measured per window, and the cliff can arrive suddenly.
4. Print the truncated count next to every pass rate. A model that runs out of tokens is not a model that writes bad code, but it is still a model that does not finish your task.
5. Set a speed floor before you compare scores. Below about 10 tok/s a model is a batch tool, and below about 20 tok/s it is hard to use interactively.
6. Treat gaps below about 3 points as noise, and use a paired test on the same problems when you need to separate two close models.

...

Limits of this setup

HumanEval has 164 short problems, so small differences stay hidden, and many models have probably seen these problems during training. Every run here is a single pass at temperature 0, so it measures one sample per problem, not the spread across samples. Everything was measured on one 16 GB GPU with one version of LM Studio, so the numbers will shift on other hardware. The windows were tested with short prompts, so a full window may cost more speed than shown here. And I did not test KV cache quantization, which LM Studio's load API does not expose. Treat the method as the result you carry over, and the numbers as a snapshot.

...

Wrapping up

For a 16 GB card, GPT-OSS-20B is the model I would run. It is the most accurate model in the study, it decodes four times faster than the next model with a similar score, and its memory use stays comfortable up to 131k tokens. The minimum speed for practical coding is about 10 tok/s for batch work and about 20 tok/s for interactive use. Below that, the model may be accurate, but you will spend your afternoon watching it type.

The method matters more than my results. Do the arithmetic first, measure speed next to accuracy, look at memory across the window you need, and check why answers failed before you trust a score. Then rank the models by what you can actually use on your hardware.

A model is only as good as what it can do on your hardware, in the time you are willing to wait.

I hope this method helps you decide faster which of next week's models deserves a download. Run it on your own machine, because that is the fastest way to internalize the ideas.

Be safe. Stay frosty.
Cheers, E.

If you found this article helpful, you are welcome to **follow me** because I write more about data science and local AI! Also try the hands-on examples; that is the fastest way to learn quicker, understand better, and remember longer. Grab a coffee and have fun!

Software

- LM Studio, to load and serve the local models
- `llm_coding_benchmark.py`, the benchmark and speed probe
- `analyze_slowdown.py`, the slowdown report

Let's connect!

Let's connect on LinkedIn

Follow me on GitHub

Follow me on Medium

References

1. M. Chen et al., Evaluating Large Language Models Trained on Code (HumanEval), arXiv:2107.03374, 2021.
2. E. B. Wilson, Probable inference, the law of succession, and statistical inference, Journal of the American Statistical Association, 1927.

3. LM Studio documentation, REST API for loading and unloading models.